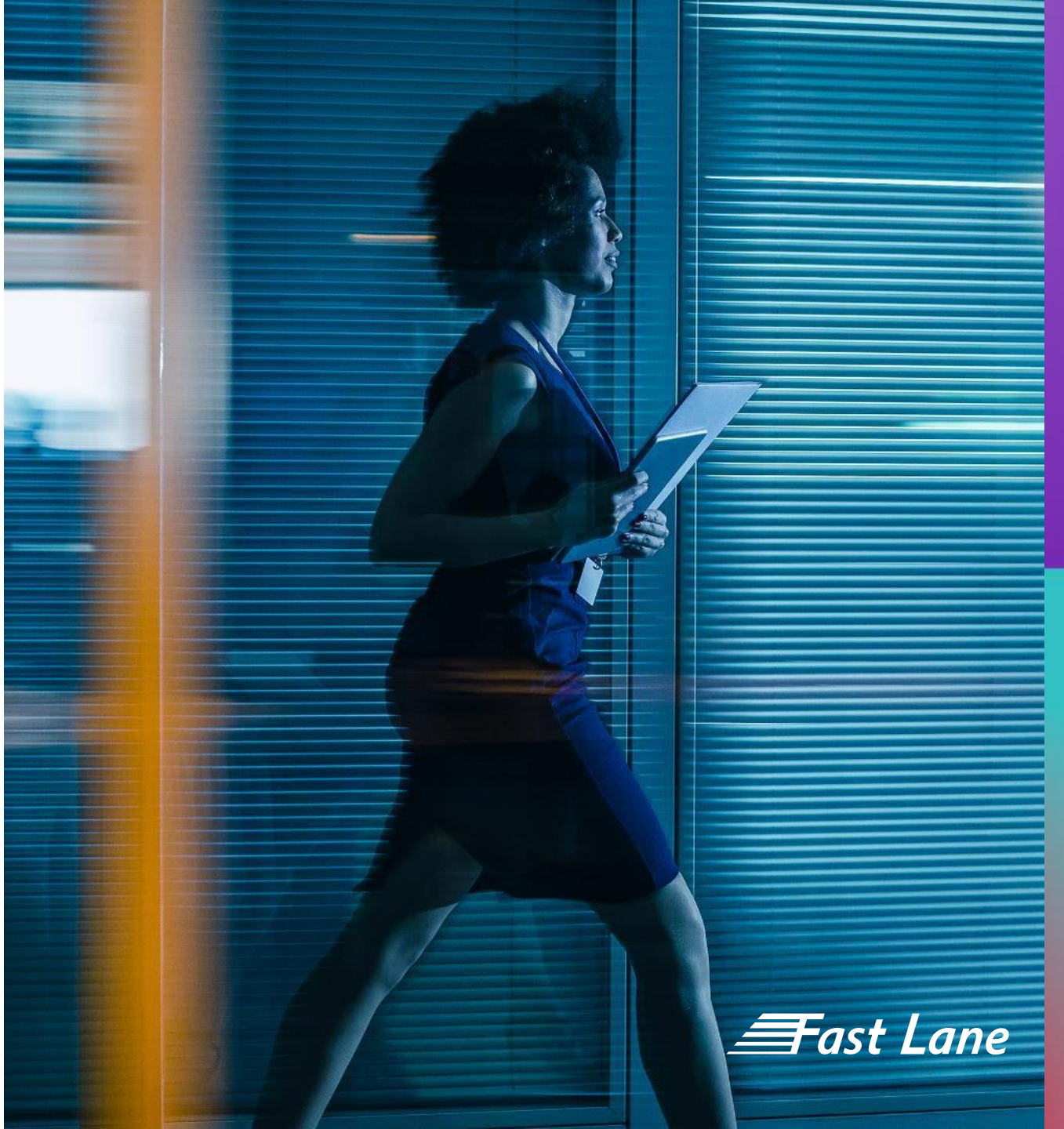
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 8

Bundle and Distribute Your Config as a Plugin

Bundle a working config and ship it to the whole team



01

Why Plugins Exist

The distribution problem a .claude/ folder cannot solve

02

Plugin Anatomy

One manifest and the components around it

03

From .claude/ Config to Published Plugin

Package, test locally and tag a release

04

Marketplaces and Distribution

Catalogs, pinning, policy and safe installs

05

Plugins and Repo Rules

The repo half of every global skill and agent

Plugins and marketplaces

Bundle a working config and ship it to the whole team

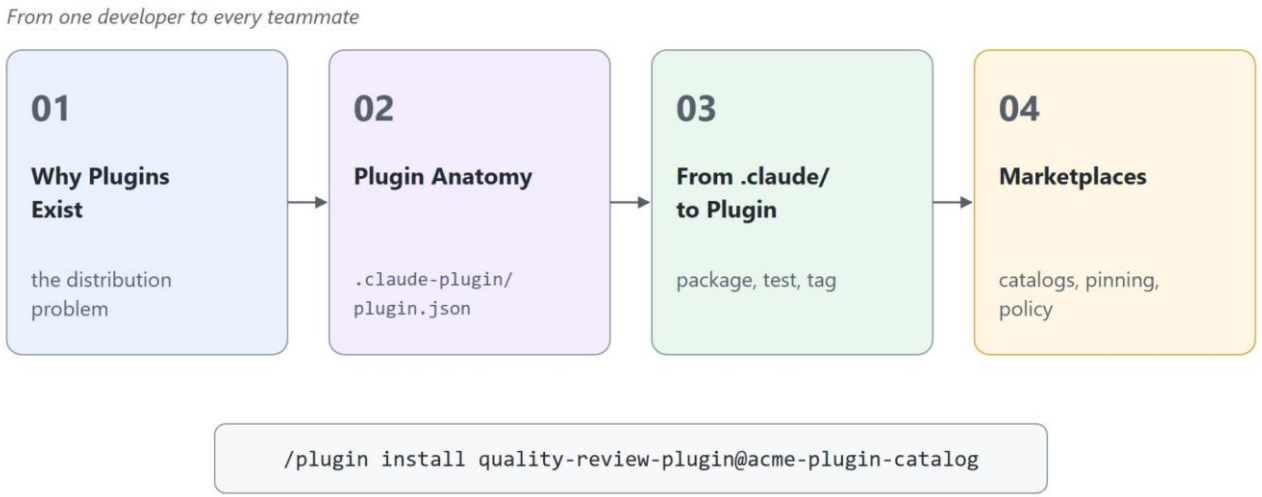
- **Why plugins**: a `.claude/` folder has no name, no version and no install command
- **Anatomy and conversion**: one manifest, the component tree, and the move from `.claude/`
- **Distribution**: marketplace catalogs, pinning, enterprise controls and security



Overview

The path from a local config to a team install

- Start with the problem a standalone `.claude/` folder cannot solve
- Read the anatomy: one required **manifest**, every other component optional
- Convert a working config, test it with `--plugin-dir`, cut a release tag
- Publish through a **marketplace catalog** and govern installs with managed policy



01

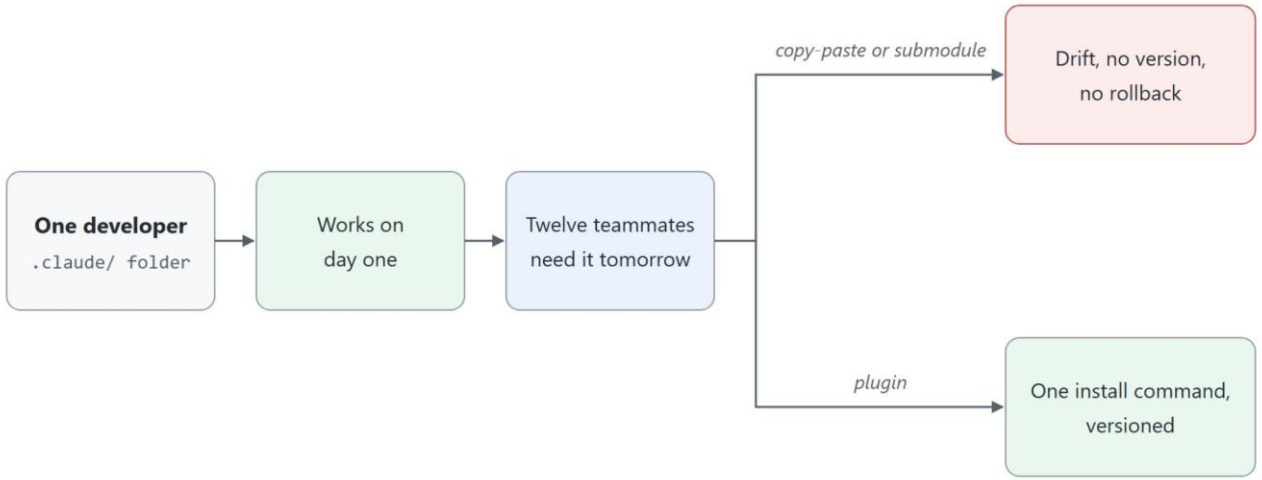
Why Plugins Exist

The distribution problem a .claude/ folder cannot solve

The distribution problem

A folder that works for one developer and drifts for twelve

- Skills, hooks, MCP configs and subagents in `.claude/` serve **one developer**
- Copy-paste and git submodules have **no version**, no install command, no rollback
- A plugin turns the folder into one **installable unit** with a name and a version
- Roll back means installing the previous version, not repairing a teammate's copy



Plugin or standalone config

Same capabilities, a different distribution model

- Both forms support **skills, hooks, MCP servers** and subagents
- Standalone iterates faster; a plugin adds **semver**, one install command and marketplaces
- The one user-visible cost: `/hello` becomes `/my-plugin:hello`
- Prototype in `.claude/`, convert the day you need **versioning or sharing**

	`.claude/` standalone	Plugin
Iteration speed	faster, edit in place	slightly slower, reload step
Versioning	none, whatever is on disk	semver via manifest
Sharing	copy-paste or git submodule	one install command
Skill namespace	`/hello`	`/my-plugin:hello`
Marketplace	no	yes

the only real cost

02

Plugin Anatomy

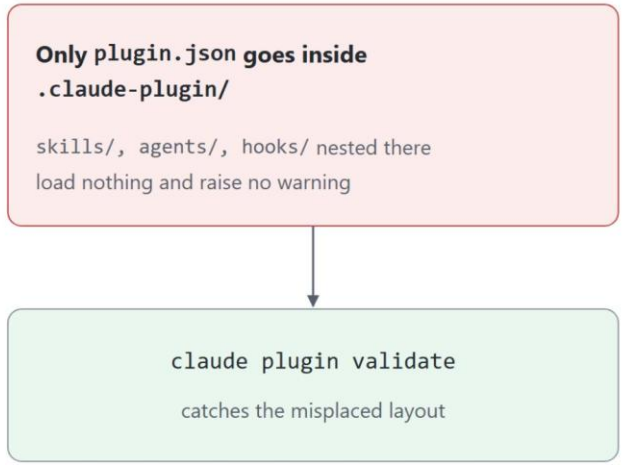
One manifest and the components around it

Plugin anatomy

One required manifest, every other component optional

- A plugin is a directory with one required file: `.claude-plugin/plugin.json`
- Skills, agents, hooks, `.mcp.json`, `.lsp.json`, monitors and `bin/` are optional
- Components live at the **plugin root**, never inside `.claude-plugin/`
- Misplaced components load nothing and raise no warning: run `claude plugin validate`





The plugin manifest

Name, version and the namespace every skill inherits

- Short and declarative: **name**, **version**, description, author, repository, license
- The `name` doubles as the skill namespace: `/quality-review-plugin:hello`
- Repository and author make the plugin **auditable**; license states the terms
- Pick a name that is unique and stable: renaming it later breaks every installer



LSP servers, monitors and plugin settings

Components that go beyond a local .claude/ folder

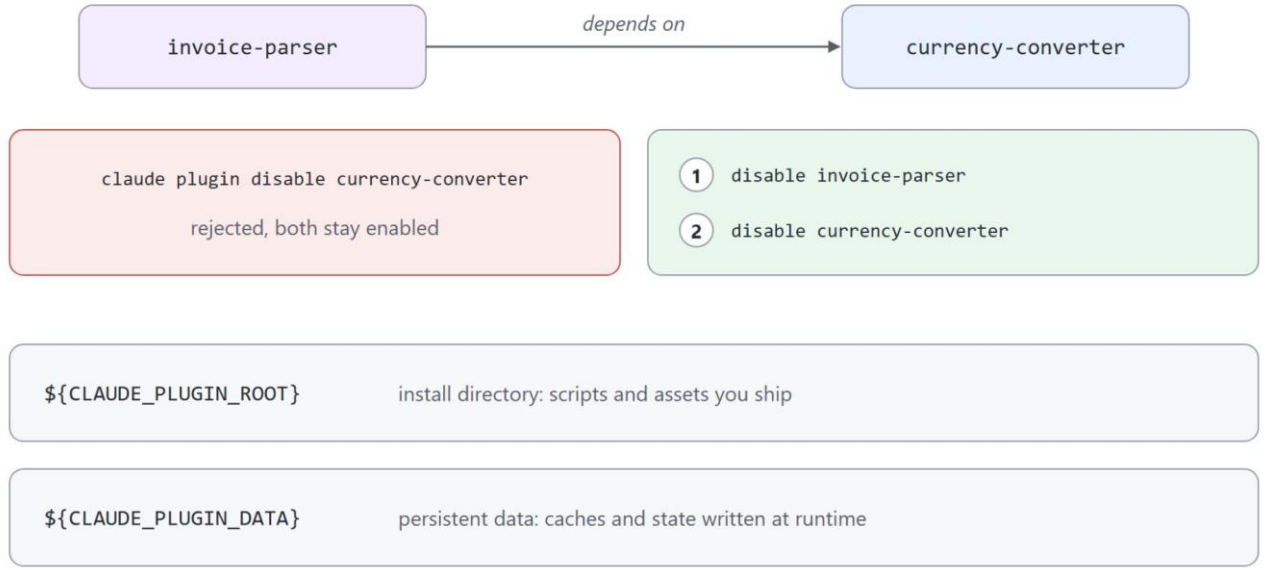
- `.lsp.json`: jump-to-definition and diagnostics for the language the agent works in
- `monitors/monitors.json`: watchers that arm when the plugin is enabled or a skill runs
- Plugin-root `settings.json` honours only `agent` and `subagentStatusLine`
- Model, permission and env keys there are **silently ignored**: keep them in `.claude/settings.json`



Dependencies and plugin paths

Disable order and the two plugin variables

- Claude Code **enforces** declared dependencies between plugins
- Disabling a dependency of an enabled plugin **fails**; nothing cascades
- Disable the dependent first, then its dependency; uninstalls follow the same order
- `${CLAUDE_PLUGIN_ROOT}` for what you ship, `${CLAUDE_PLUGIN_DATA}` for what you write



03

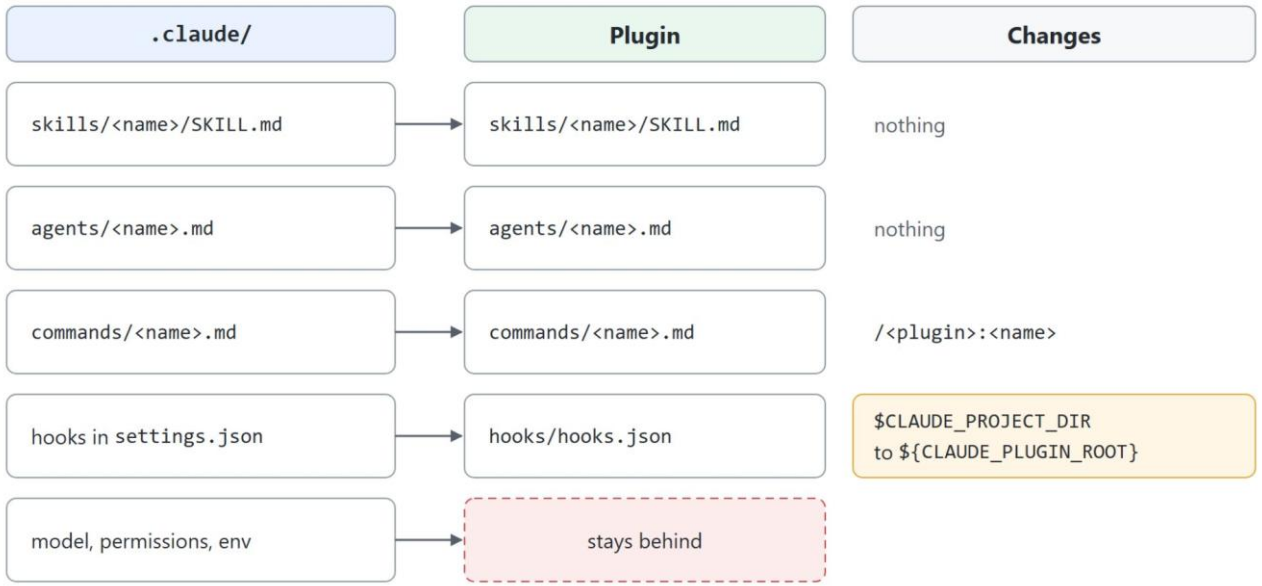
From .claude/ Config to Published Plugin

Package, test locally and tag a release

Moving a .claude/ config into a plugin

What moves unchanged and what changes form

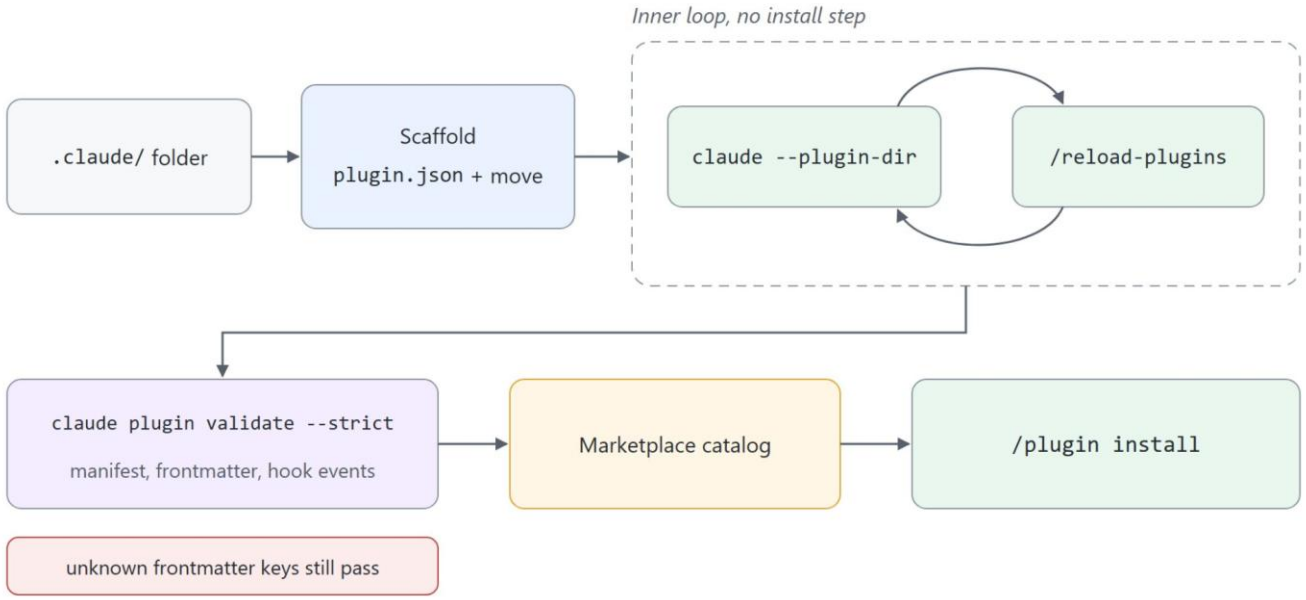
- A **packaging change**, not a rewrite: components keep filenames and contents
- Skills and agents lift one to one; commands gain the **`/<plugin>:<name>`** namespace
- Hook entries move from `settings.json` into **`hooks/hooks.json`**
- Hook paths change from `$CLAUDE_PROJECT_DIR` to **`${CLAUDE_PLUGIN_ROOT}`**
- Model, permission and env keys **stay behind** in the project



Testing a plugin locally

The --plugin-dir loop, then validation before publishing

- `claude --plugin-dir` loads the plugin from disk, no install, no catalog
- `/reload-plugins` picks up a component edit in place
- `claude plugin validate` catches manifest errors, broken frontmatter, bad hook events
- Unknown frontmatter keys pass validation: a human still reads the frontmatter
- Publish to a catalog once the plugin works



Release tags and version identity

The tag comes from plugin.json alone

- `claude plugin tag` cuts `{name}--v{version}` from `plugin.json` alone
- The catalog's version is **never cross-checked**: bump both in one commit
- Tagging refuses while anything under the plugin path is **uncommitted**
- Pin installers to a **tag** or a full **sha**, not a moving branch



Package, validate and publish a plugin

The demo in three steps

- **Scaffold**: add `plugin.json`, copy skills and hooks, rewrite the hook to `${CLAUDE_PLUGIN_ROOT}`
- **Inspect**: `claude plugin validate --strict` and `plugin details`
- **Publish**: an `acme-plugin-catalog` with one entry, then `claude plugin tag --dry-run`



04

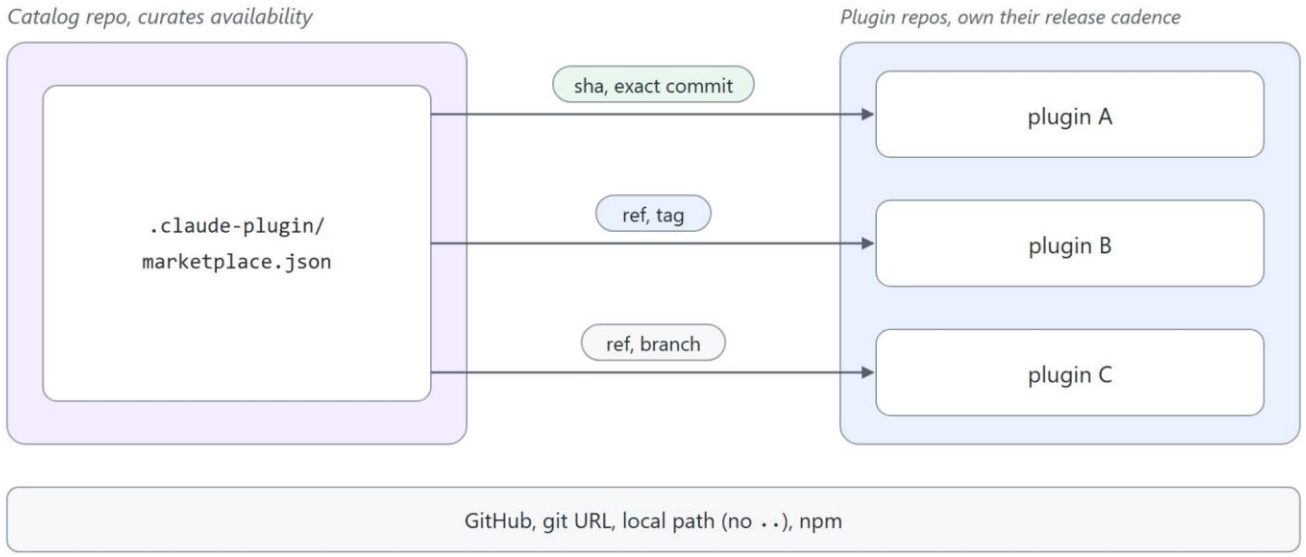
Marketplaces and Distribution

Catalogs, pinning, policy and safe installs

Catalogs and plugin repositories

One catalog curates plugins from many repositories

- Catalog source and plugin source are **separate repositories**
- One `marketplace.json` lists plugins from many repos, each pinned on its own
- **`ref`** follows a branch or tag; **`sha`** installs one exact commit
- Sources: GitHub, git URLs, npm, or a local path that may not contain `..`



Plugins by Anthropic

The official marketplace and the community catalog

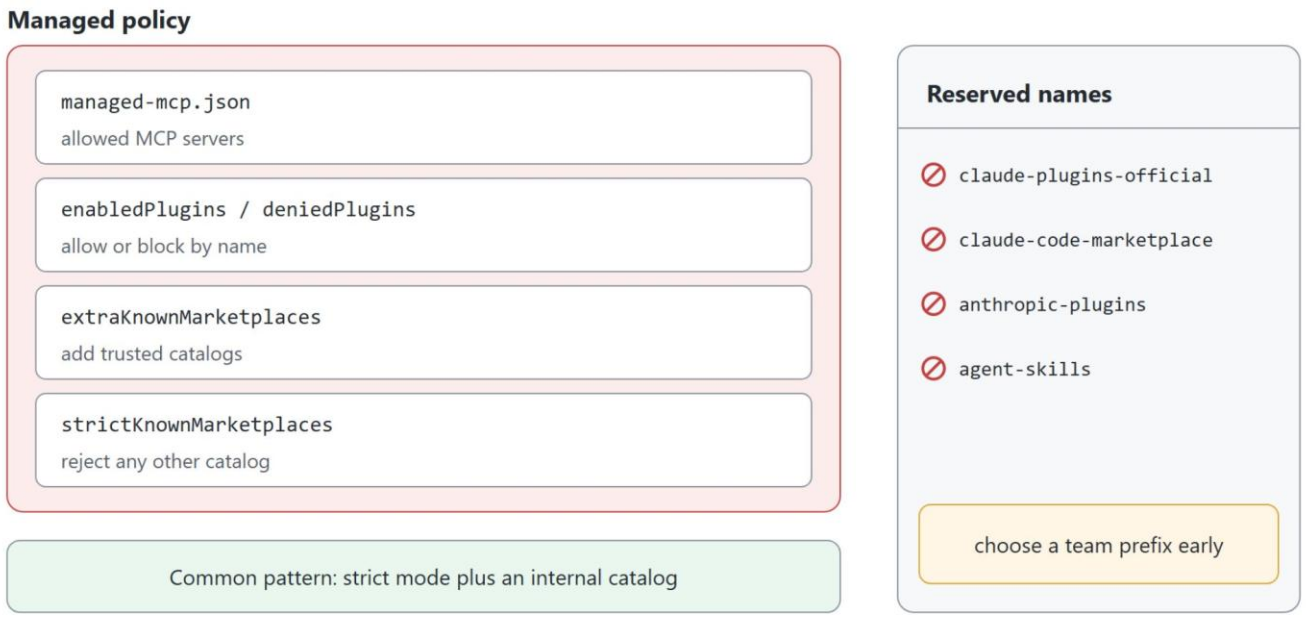
- `claude-plugins-official` registers itself on the first interactive start
- `/plugin install <name>@claude-plugins-official`, with a context-cost estimate per plugin
- `plugin-dev`, `hookify`, `skill-creator`, `pr-review-toolkit`, `security-guidance`
- Screened third-party plugins: `anthropics/claude-plugins-community`
- Submit to the official directory at `claude.ai/directory/manage/new`



Enterprise controls and reserved names

Managed policy decides what teams can install

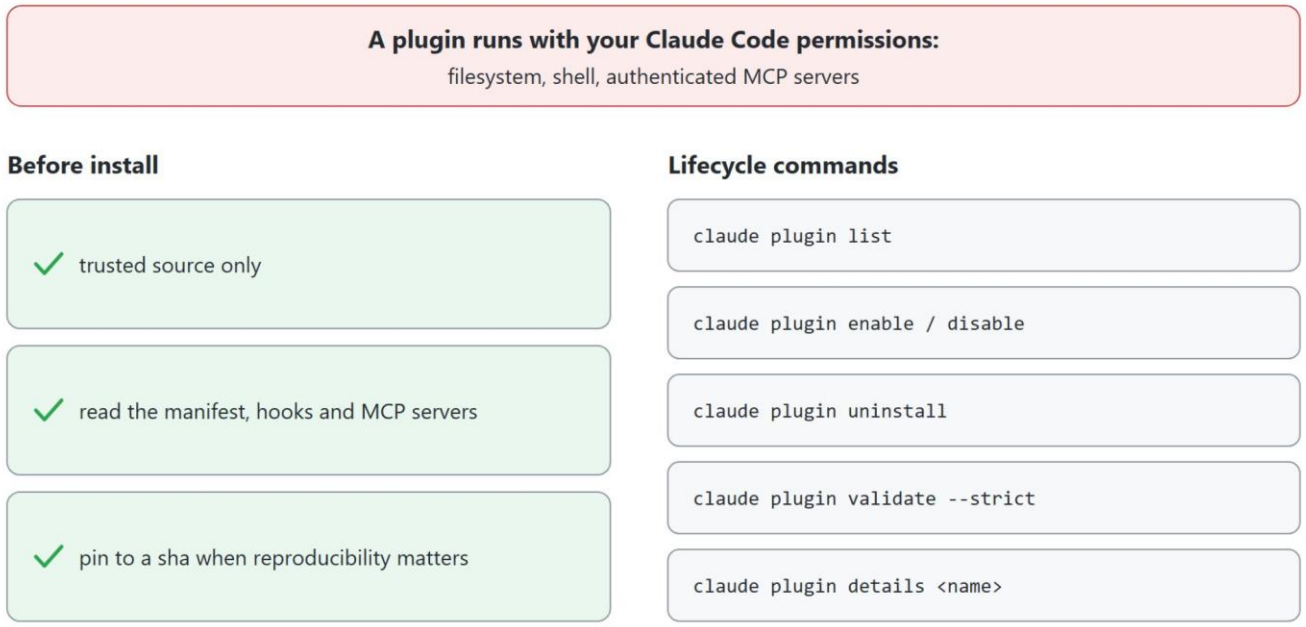
- `managed-mcp.json` controls which MCP servers the organization allows
- `enabledPlugins` and `deniedPlugins` allow or block plugins by name
- `extraKnownMarketplaces` adds trusted catalogs; `strictKnownMarketplaces` rejects the rest
- Names that impersonate Anthropic distribution are **reserved**: pick a team prefix early



Installing plugins safely

Plugins run with your permissions

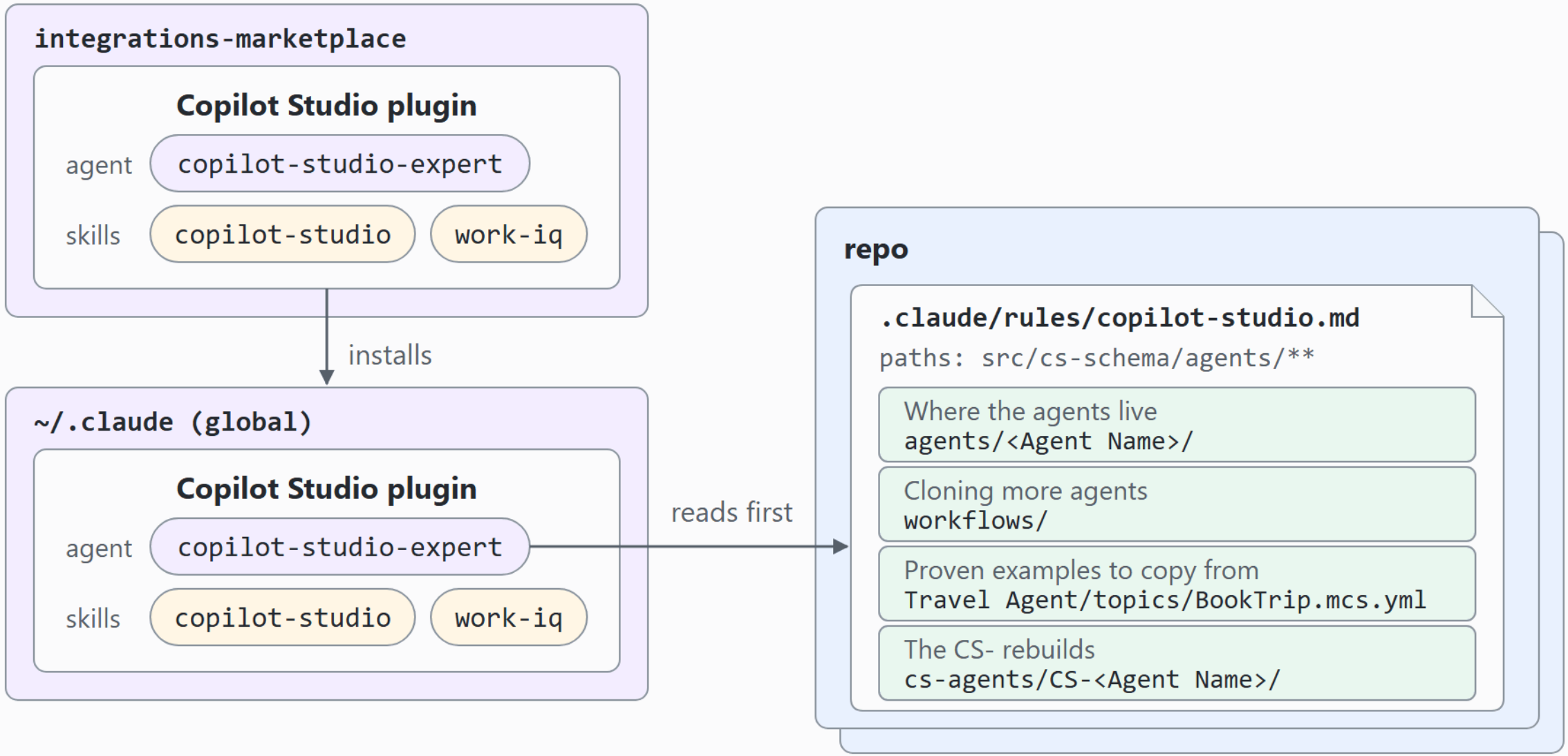
- A plugin inherits your **filesystem, shell** and authenticated MCP servers
- Install from **trusted sources** and read the manifest before approving
- Pin to a **sha** when reproducibility matters more than convenience
- `claude plugin disable` toggles a misbehaving plugin without uninstalling it
- `/permissions` and `/mcp` show what an installed plugin can reach



05

Plugins and Repo Rules

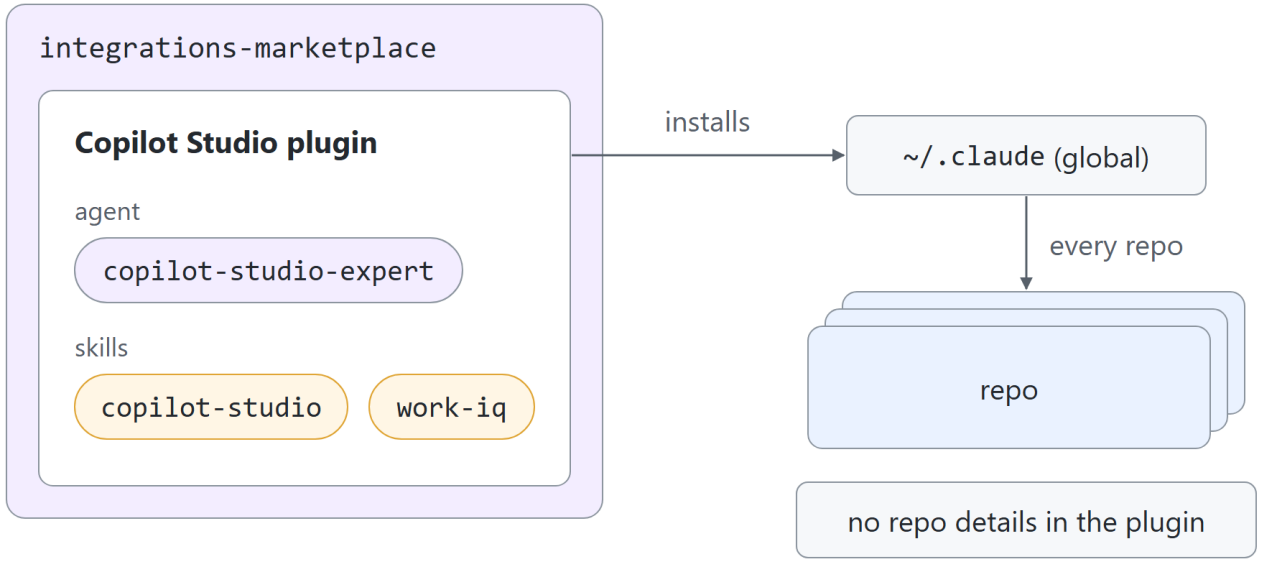
The repo half of every global skill and agent


integrations-marketplace
Copilot Studio plugin
agent
copilot-studio-expert
skills
copilot-studio
work-iq
installs
~/.claude (global)
Copilot Studio plugin
agent
copilot-studio-expert
skills
copilot-studio
work-iq
reads first
repo
.claude/rules/copilot-studio.md
paths: src/cs-schema/agents/**
Where the agents live
agents/<Agent Name>/
Cloning more agents
workflows/
Proven examples to copy from
Travel Agent/topics/BookTrip.mcs.yml
The CS- rebuilds
cs-agents/CS-<Agent Name>/

A marketplace plugin brings its own expert

One expert for every repo, with no repo details of its own

- The **Copilot Studio plugin** from the team marketplace ships the `copilot-studio-expert` agent and its skills
- It installs once, **globally**; the same expert then works in **every repo**
- No plugin component carries repo details, and a plugin-root `CLAUDE.md` is not loaded



The repo hands the expert its details

One rule, scoped to the folders the expert works in

```
---
paths:
  - "src/cs-schema/agents/**"
  - "src/cs-schema/cs-agents/**"
---

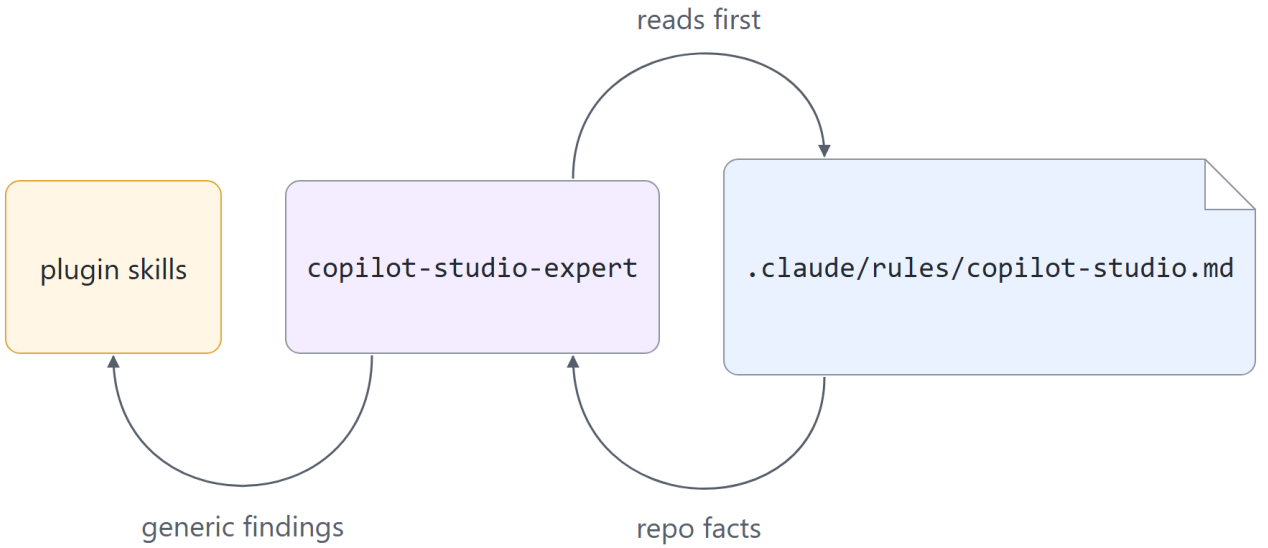
# Copilot Studio agents in this repo
```



The expert reads the rule first

Repo facts stay in the rule, generic findings go back to the plugin

- The expert's definition reads `.claude/rules/copilot-studio.md` before it touches agent folders
- A path-scoped rule loads only on a matching file, so the expert **reads it explicitly**
- **Repo facts** stay in the rule; **generic findings** go back into the plugin's skills



Takeaways

What to carry into the next module

- Prototype in `.claude/`, convert to a plugin the day you need **versioning or sharing**
- Only **`plugin.json`** goes in `.claude-plugin/`; validate before every publish
- Ship scripts under **`${CLAUDE_PLUGIN_ROOT}`**, write state under **`${CLAUDE_PLUGIN_DATA}`**
- Bump `plugin.json` and `marketplace.json` together; pin production to a **sha**
- Gate installs with **managed policy** and an internal catalog

